Alert2Vec: Eliminating Alert Fatigue by Embedding Security Alerts Through Subgraph Learning

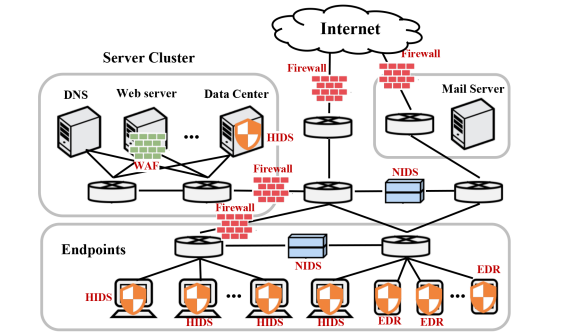
Songyun Wu, Xiaoqing Sun, Enhuan Dong, Zhiliang Wang, Chen Zhao, and Jiahai Yang, *Senior Member, IEEE*

***Abstract*—With increasingly sophisticated attacks in cyberspace, a growing number of security devices are deployed on the organization network to prevent attacks comprehensively. However, they produce thousands of security alerts each day, causing a severe burden on security analysts. Thus, it is increasingly concerned about capturing the significant threats from the jumble of alerts. Previous works only provided scene-specific algorithms for given applications or services, which is hard to extend to other scenarios and consequently not adequate for large organizations that deploy multiple kinds of security devices. To address this, we propose a general alert classification approach named Alert2vec. With subgraph learning methods on the Heterogeneous Information Network (HIN), Alert2vec is applicable for both network-level and host-level security devices. Specifically, Alert2vec first builds a heterogeneous Alert Intelligence Graph (AIG) to catch the correlation between different alerts. Then, a novel subgraph representation method is applied to generate alert vectors based on six graph properties that can reveal the threat level of alerts. Finally, these alert vectors are labeled with various threat levels in semi-automatic or fully automatic classification mode. Alert2vec is proven to be effective on real-world datasets, outperforming all baselines on both network-level and host-level scenarios.**

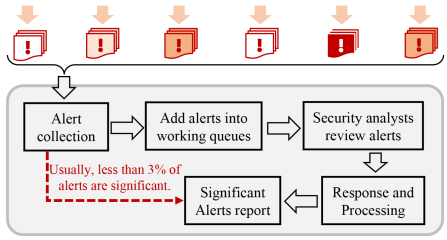
***Index Terms*—Alert fatigue, threat alerts, subgraph learning.**

I. Introduction

To combat increasingly frequent and complex attacks, large organizations tend to deploy multiple security devices on their computer systems and networks. These security devices constantly collect various monitoring data and trigger an alert once a suspicious activity corresponds with the preset detection rules. Fig. 1(a) depicts an example of security defenses for an enterprise network. In front of significant network infrastructures, some *network-level* detection devices are deployed to check the network data flow, such as Firewall, Web Application Firewall (WAF), and Network Intrusion Detection System (NIDS). Simultaneously, a series of *host-level* detectors are applied to monitor process activities in servers and endpoints, e.g., Endpoint Detection and Response (EDR), Host Intrusion Detection System (HIDS), and anti-virus software. As shown in Fig. 1(b), security analysts need to review various alerts generated by these devices one by one to identify and report severe alerts.



(a) An example of the deployment of security devices on an enterprise network.



(b) The workflow of alert analysis.

Fig. 1. An example of enterprise network's security defenses and alert processing. (a) illustrates that enterprise networks typically deploy numerous security detection devices. (b) demonstrates that the alerts generated by these devices require manual analysis and processing, which is both complex and time-consuming.

However, these security devices frequently generate massive low-severity alerts, leading to *alert fatigue* among security analysts [1], [2]. As illustrated in Fig. 2, a typical campus network produces over 1,000 daily alerts, with 97% classified as low- or medium-risk. Furthermore, the threat assessment process for each alert is highly time-intensive (Fig. 1(b)). Consequently, security teams face the formidable challenge of identifying major threats amidst this overwhelming volume of alerts.

To address this, an automatic approach to classify alerts is urgently required, i.e., identify significant threats from enormous alerts. While the security community has drawn attention to

Received 13 November 2023; revised 16 July 2025; accepted 7 September 2025. Date of publication 15 September 2025; date of current version 14 January 2026. This work was supported in part by the National Key R&D Program of China under Grant 2022YFB3102902 and in part by Scientific Research and Development Plan Project of China Railway Information Technology Group Co., Ltd. under Grant WJZG-CKY-2024005(2024K02). *(Corresponding authors: Enhuan Dong; Jiahai Yang.)*

Songyun Wu, Xiaoqing Sun, Enhuan Dong, Zhiliang Wang, and Jiahai Yang are with the Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing 100190, China (e-mail: shoinwu@gmail.com; sxq16@mails.tsinghua.edu.cn; dongenhuan@tsinghua.edu.cn; wzl@cernet.edu.cn; yang@cernet.edu.cn).

Chen Zhao is with the Information Technology Department of Asian Development Bank, Mandaluyong City 1550, Philippines (e-mail: czhao@adb.org).

Digital Object Identifier 10.1109/TDSC.2025.3609834

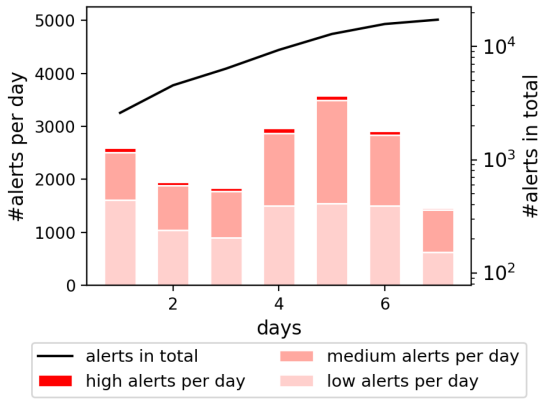


Fig. 2. The number of alerts in a network-level security dataset produced by the campus network of Tsinghua University during a week. The red bars stand for the number of alerts produced per day, with the left vertical axis as the reference. The proportion of alerts with higher threat levels is darker in color. In addition, the black curve represents the cumulative number of alerts, with the right vertical axis as a reference.

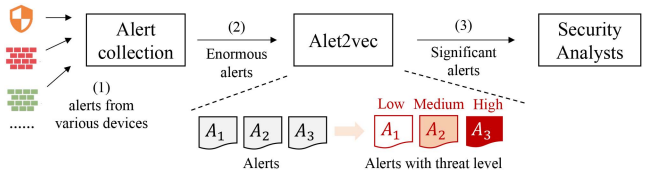


Fig. 3. An illustration of how Alert2vec alleviates the burden on security analysts in alert processing. Alert2vec aims to automatically assess threat levels in voluminous multi-source alert logs and prioritizes critical alerts for security analysts.

eliminating alert fatigue, recent works merely focused on a certain scenario at the host-level [1], [3] or the network-level [2]. Hassan et al. [1] used a heuristic network diffusion algorithm to compute each alert's anomaly score in a dependency graph generated from audit logs. Obviously, this work cannot be extended to the network-level detectors since it needs plenty of fine-grained host audit logs to build a vast provenance graph (a graph that combines all entities, control flows, and information flows in audit logs). Likewise, Zhao et al. [2] leveraged several scene-specific features, such as KPI metrics, to catch severe alerts, which is only appropriate for the online service system and hard to extend to other scenarios. However, as Fig. 1(a) presents, large organizations tend to deploy a variety of different security devices to protect network communication and host activities comprehensively. Unfortunately, these single-scenario-oriented works cannot be applied to both network-level and host-level detectors simultaneously. Thus, a naïve way is to deploy an alert classification approach for each detector, which remains two problems. (i) It costs highly in terms of labor and time, and even no existing method is available for some devices. (ii) It ignores the correlation between alerts generated from various devices, which can provide valid information to recognize false positives [4]. Considering that these works are insufficient to eliminate alert fatigue for large organizational networks, there is an imperative need for a general alert classification method.

Hence, we aim to propose an intelligence alert classification framework applicable to reduce the workload of security analysts in large organizations, as given in Fig. 3. Between enormous alerts and exhausted security analysts, an alert-processing buffer is created for classifying heterogeneous alerts into several threat levels based on the correlation between alerts. Then, instead of facing tremendous alerts before, security analysts can just focus on significant alerts with a small percentage. To this end, there are three challenges to the approach. (i) **Generality.** The compatibility for heterogeneous alerts generated by various devices and the adaptability to different organizational environments with diverse device deployments. (ii) **Alert correlation.** The ability to reasonably express the correlation between alerts and leverage it to identify threat levels. (iii) **Less Labor.** Since the primary purpose is to alleviate the burden of security analysts, the approach should introduce as little labor as possible, namely, having a high level of automation and interpretation ability.

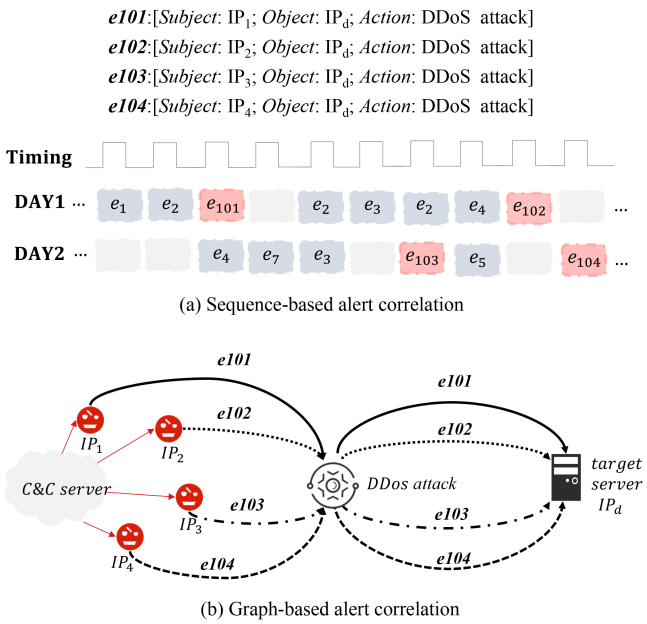
In summary, we propose Alert2vec, a general alert classification framework addressing the three aforementioned challenges. To resolve the first two challenges, we extract three generic elements (*subject*, *object*, and *action*) to construct a subgraph for each alert. This design ensures compatibility with diverse alerts from both network-level and host-level security devices in large networks. These subgraphs are unified into a heterogeneous Alert Intelligence Graph (AIG), where correlations are inferred via shared nodes (i.e., identical alert elements) at a fine-grained level. For the third challenge, we introduce a novel subgraph learning method to embed alert subgraphs into discriminative vectors for automated threat-level identification. The embedding process optimizes six subgraph properties, capturing both global and local correlation features. Finally, threat levels are classified via a dual-mode approach (semi-automatic and fully automatic), leveraging the derived alert vectors.

Alert2vec is demonstrated to be effective on three real-world large organization's datasets, outperforming all baseline methods on both network-level and host-level scenarios. To conclude, this paper makes the following contributions:

- We design a heterogeneous graph named AIG and propose a general framework that can classify alert threat levels for both network-level and host-level security devices.
- We devise a novel subgraph learning method for representing alert subgraphs. By considering six graph properties, it can comprehensively extract both the global and local information of subgraphs.
- Alert2vec is proven to be effective on three real-world datasets, including the network-level dataset of Tsinghua University and the host-level dataset of Asian Development Bank (ADB), and a publicly available network-level dataset.

II. Motivation

We motivate the design of Alert2vec by investigating (i) the weakness of current alert correlation and (ii) the essentiality of a subgraph learning method for threat level identification.



(a) Sequence-based alert correlation

(b) Graph-based alert correlation

Fig. 4. The motivation example of sequence-based and graph-based representation of an attack that *a botnet C&C server communicating with multiple controlled hosts over a long period*, which consists of four alerts, i.e., ***e101***, ***e102***, ***e103***, and ***e104***.

A. Why Current Alert Correlation is Insufficient?

In order to present drawbacks of existing works, we introduce a motivation example first. From a fine-grained perspective, a security alert comprises three main elements, i.e., the *subject*, *object*, and *action*. In NIDS, they can be source IP address, destination IP address, and alert type (such as *SQL injection attack*.), respectively. As Fig. 4 depicts, consider an attack of *a botnet C&C server communicating with multiple controlled hosts over a long period* that may trigger multiple alerts, including **e101***(subject: IP_1; object:IP_d; action:DDoS attack)*, **e102***(subject: IP_2; object:IP_d; action:DDoS attack)*, **e103***(subject: IP_3; object:IP_d; action:DDoS attack)*, and **e104***(subject: IP_4; object:IP_d; action:DDoS attack)*.

To catch the correlation among these alerts, current works commonly leverage a sequence-based method, i.e., converting alerts into security events and capturing the contextual relationship in an event sequence arranged chronologically, as shown in Fig. 4(a). For example, DeepCase [4] aggregates similar security events generated from network security monitors through the contextual relationship in sequences. However, it still strongly depends on the manual judgment of the risk level, introducing a new labor-intensive work for security analysts. Likewise, Context2vector [5] assigns an anomaly score to a security event based on its deviation from the context in the sequence. It extremely relies on the correctness of the previous and subsequent events in the sequence, which inevitably contains plenty of unreliable false alerts.

Sequence-based methods have limited expressiveness in capturing alert correlations, as they only model linear event sequences while neglecting complex inter-alert relationships. As shown in Fig. 4(a), botnet-related alerts often appear scattered across disjointed fragments due to covert communication patterns, making it difficult to infer their collective role in an attack. However, when analyzed by target (e.g., IP_d) and action (e.g., DDoS attack), these alerts can be intuitively aggregated (Fig. 4(b)). To better represent such correlations, we propose a graph-based approach, encoding each alert as an alert graph with triplet elements as nodes and their relationships as edges. Furthermore, we aggregate all alert graphs into an Alert Intelligence Graph (AIG) to comprehensively model inter-alert dependencies (see Section III). This framework captures both individual alert features and fine-grained element-level correlations.

B. Why is a Subgraph Learning Method Expected?

As shown in Fig. 4(b), graph-based alert representations reveal that related alerts from the same attack source form dense clusters. Thus, an alert's threat level depends on both local (intrinsic features) and global (AIG correlations) properties. Since each alert is a variable-sized subgraph of AIG, specialized subgraph learning is needed to capture these properties. Existing subgraph methods focus on homogeneous graphs [6], [7], [8] or fixed structures [9], ignoring: (i) Heterogeneity: Our scenario involves three distinct node types (subject, object, action); (ii)Dynamic topology: Alert subgraphs vary in size/structure; (iii)Security-specific features: Current methods lack threat-level classification capabilities (see Section IV). Thus, this necessitates a tailored subgraph learning approach.

III. System Framework

In this section, we first introduce the problem formulation, and then the alert classification framework as presented in Fig. 5, including alert intelligence graph construction, alert vector generation, and threat level identification.

A. Problem Formulation

Definition 1: **(Alert Subgraph, AS)** The heterogeneous security alert records from various devices will be normalized into alert subgraphs by extracting three kinds of generic elements, i.e., subject, object, and action. Each alert record can be converted into an alert subgraph represented as $A = (V_A, E_A)$, where V_A is the node set and E_A is the edge set. For each node $u \in V_A$, it stands for a representative feature extracted from the alert. Moreover, the element type of each node $T(u) \in \{subject, action, object\}$. Besides, edges in the alert subgraph mean inner correlation among its elements or the contextual relationship, the type of each edge $T(e) \in \{(subject, action), (action, action), (action, object)\}$. In particular, the edge can be either undirected or directed according to the applying scenario, which will be detailed in Section III-C.

Definition 2: **(Alert Intelligence Graph, AIG)** Given a set composed of multiple alert subgraphs, $A_1, A_2, \ldots, A_n$, the alert intelligence graph can be denoted as $G = (V, E)$, where $V = \{V_{A_1} \cup V_{A_2} \cup \ldots \cup V_{A_n}\}$, $E = \{E_{A_1} \cup E_{A_2} \cup \ldots \cup E_{A_n} \cup E_C\}$, and n is the number of alert subgraphs. It denotes that AIG is the union of all alert subgraphs and

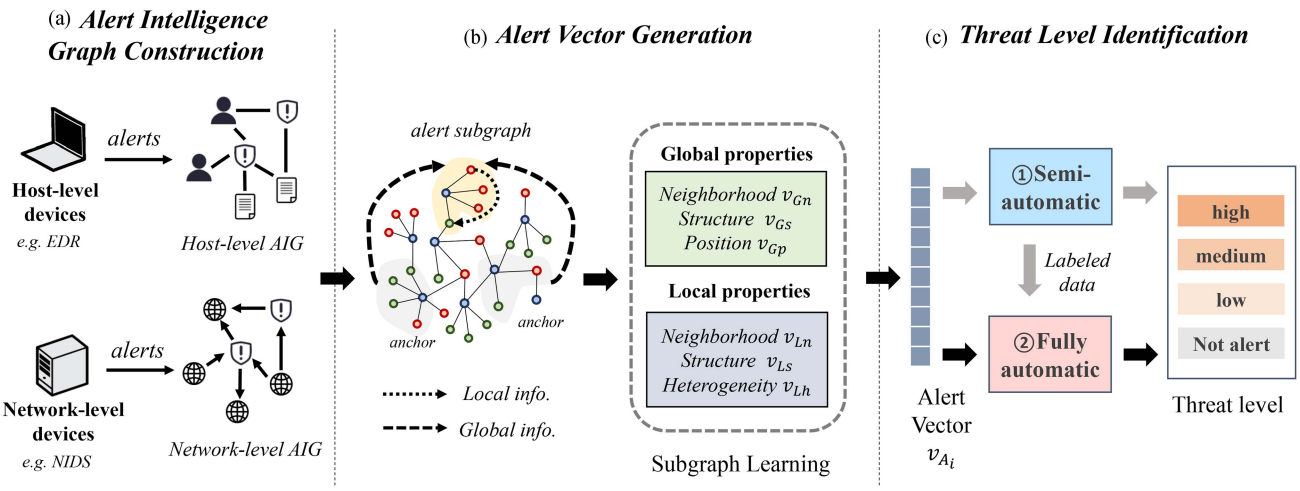


Fig. 5. The framework of Alert2vec contains three phases: (a) Alert intelligence graph construction: heterogeneous alerts are transformed into subgraphs and aggregated into network/host-level AIGs; (b) Alert vector generation: we propose a six-property subgraph learning method capturing both local (*neighborhood*, *structure*, *heterogeneity*) and global (*neighborhoo*, *structure*, *position*) features; (c) Threat level identification: threat levels are classified via semi/fully-automatic modes using the learned vectors, enabling analysts to focus on critical alerts.

additionally incorporates a new edge set E_C. E_C contains the edges connecting two *contextual alerts*. Once two alerts that occur within a short period (e.g., 1 s) have a common element in subject or object nodes, they are regarded as contextual alerts. The edge between two contextual alert subgraphs' action nodes will be added into E_C. Thus, for each edge e_c in E_C, its edge type $T(e_c) = (action, action)$.

Definition 3: **(Alert Classification)** The problem we aim to tackle is identifying the threat level of each alert subgraph with the aid of alert intelligence graph G. The formulaic exposition is presented as follows.

$$\hat{l}_i = F\left(H\left(A_i, G\right)\right) \tag{1}$$

where $\hat{l}_i$ is the targeted threat level of alert subgraph A_i, F is a nonlinear mapping function, and H is a function that transfers the alert subgraph A_i into a numerical vector.

Consequently, in order to derive the threat level result $\hat{l}_i$, three issues need to be addressed, namely, (i) the construction of graphs A_i and G, (ii) the construction of transfer function H, and (iii) the design of mapping function F. The following sections will elaborate on corresponding solutions.

B. Overview

The framework of Alert2vec (Fig. 5) includes three modules: (i) AIG construction, (ii) Alert vector generation, and (iii) Threat level identification. First, heterogeneous alerts are transformed into subgraphs and aggregated into network/host-level AIGs (Section III-C). Second, we propose a six-property subgraph learning method capturing both local (*neighborhood*, *structure*, *heterogeneity*) and global (*neighborhoo*, *structure*, *position*) features (Section IV). Finally, threat levels are classified via semi/fully-automatic modes using the learned vectors, enabling analysts to focus on critical alerts (Section III-E).

C. Alert Intelligence Graph Construction

A major reason for alert fatigue is that most existing security devices utilize simple matching rules to generate alerts [2]. Whereas, plenty of false or low threat-level alerts (such as reports of sensitive inspection operations from network managers) will also be detected by such static rules. Since these alerts are similar in form, it is hard to assess threat levels only by leveraging single alert event information.

Instead, some high-threat attacks have partial similarities or contextual relationships. For example, adversaries tend to attack the same target (e.g., *passwd* file) or leverage similar steps to compromise hosts. In addition, an adversary may conduct multiple steps successively to perform an advanced attack, and alerts triggered by these steps have contextual relations. In order to catch such relationships, alerts are expected to aggregate together. Nevertheless, merging alerts from various devices is challenging due to the heterogeneous alert reports.

Hence, a general alert normalization approach is urgently expected to accommodate heterogeneous alerts in one graph. The intuitive insight is to extract similar elements on each alert, and the ensuing question is how to pick the representative and common elements for alert classification. Through the feature engineering by Random Forest (RF) [10], three kinds of significant elements with the highest weight are selected, namely, *subject*, *object*, and *action*.

In a NIDS, *subject* and *object* can be the source and destination IP address, respectively. *action* may be the attack type, such as *SQL injection attack*. While in a HIDS, *subject* and *object* may be the process or file. Due to the distinction in alert elements between host-level and network-level security devices, the specific definition of ASs and AIGs are presented separately.

Host-level AS (HAS): The host-level security devices (e.g., EDR, HIDS) monitor fine-grained activities in hosts or endpoints. They will trigger an alert for a single abnormal event. Therefore, element *subject* points to who performs the

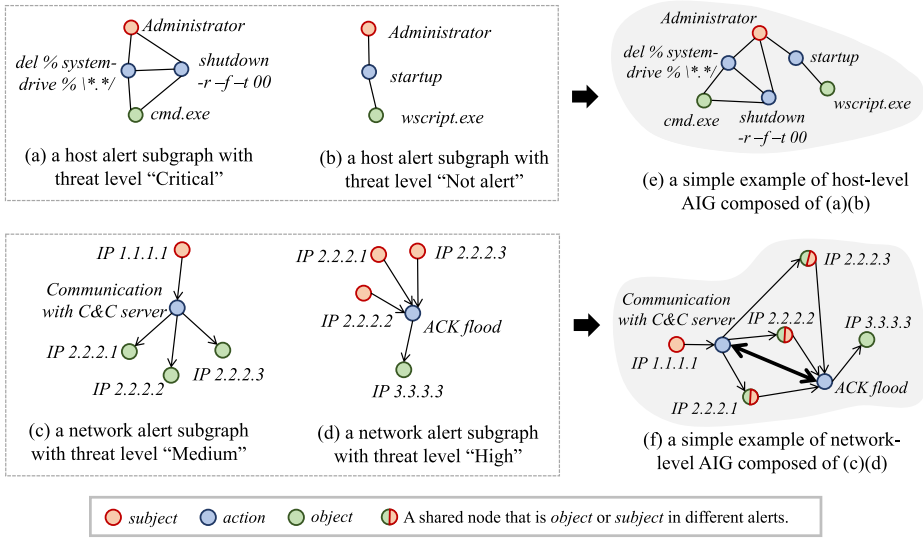


Fig. 6. Examples of host-level alert subgraphs, network-level alert subgraphs, and AIG construction. Nodes colored red are the subject elements, blue ones map to the action elements, and green ones stand for the object elements.

abnormal event, i.e., users or processes. Element *action* refers to detailed operations, e.g., creation, modification, startup, and so on. Element *object* represents the victim of this abnormal event, generally a file, a process, etc. The connection relationship of these elements in a HAS can be defined as below:

$$subject \leftrightarrow action, object \leftrightarrow action, action \leftrightarrow action$$

The first two edges refer to the inner correlation. And the last one is designed for alerts which contain multiple actions, e.g., alert in Fig. 6(a). The various actions taken by an adversary in one attack have a contextual relationship. Since there is no special dependency between each element, edges in host-level alert subgraph and AIG are undirected. Examples of HAS are shown in Fig. 6(a) and (b). It is noted that there can be multiple items in all element types, such as Fig. 6(a), which has two action nodes.

Network-level AS (NAS): The network-level security devices (e.g., Firewall, NIDS) detect abnormal network communication, the monitoring objects of which are network flows and data packets. Thus, element *subject* stands for the source IP address, and element *object* means the destination IP address. Element *action* is the malicious communication pattern, such as SQL injection, Denial of service, port scan, etc. The connection relationship of these elements in a NAS is as follows:

$$subject \rightarrow action, action \rightarrow object, action \leftrightarrow action$$

Like HAS, the last edge refers to the contextual relationship between different actions. But the first two edges are directed, since the network flows and data packets conduct the information flows, which have a natural direction from information providers to receivers. Examples of NAS are shown in Fig. 6(c) and (d).

AIG construction: First, the AIG contains the union of all HASs or NASs, as exampled in Fig. 6(e). Two HASs are connected by the same subject node, i.e., *Administrator* in the host-level AIG. Besides, in HAS and NAS, it is available to incorporate multiple items to compose an element or specify an element as "None" when the corresponding value is missing.

Second, to further capture the contextual correlation between different alerts, there is a new edge type between various alert subgraphs in AIG as defined in Definition 2. Two alerts will be regarded as contextual alerts once they happen within a short period (such as 1 s) and have a common element in subject or object nodes. Each pair of contextual alert subgraphs have corresponding edges between their action nodes in the AIG. It is noted that there are two scenarios for the common element, i.e., (i) the contextual alerts have the same subject or object node, and (ii) one node acts as subject and object, respectively, in contextual alerts.

For example, as Fig. 6(f) shows, there is an extra edge between node *ACK flood* and *Communication with C&C server* since these two NASs occur within a short period and have three identical nodes, namely, *IP 2.2.2.1*, *IP 2.2.2.2*, and *IP 2.2.2.3*. These three IPs first act as victims to communicate with the malicious C&C server *IP 1.1.1.1* to receive attack commands and then act as an attacker to launch an ACK flood attack on *IP 3.3.3.3*.

D. Alert Vector Generation

Then, this phase aims to transforming the alert graph into an expressive numerical vector through subgraph representation learning. Previous methods merely exploit partial information of the subgraph, which cannot construct a comprehensive alert vector. Consequently, we devise a six-property subgraph representation in terms of global and local perspectives, which can indicate both the alert's information and the correlation with other alerts. As Fig. 5(b) presents, the six properties are global neighborhood v_{Gn}, global structure v_{Gs}, position v_{Gp}, local neighborhood v_{Ln}, local structure v_{Ls}, and heterogeneity v_{Lh}. The reason for picking these properties and the computing

way are the critical parts of this work, which will be detailed in Section IV.

E. Threat Level Identification

The final phase is to identify the threat level when giving an alert vector. The number of threat levels varies according to the organization's needs. Some require three levels (low, medium, high), and some require four levels, as Fig. 5(c) exhibits. Hence, it can be regarded as a multi-classification problem, which can be solved by classic classification methods, such as supervised machine learning and unsupervised clustering.

Although the supervised method can achieve higher accuracy, it requires a large amount of labeled data. For some newly-started organizations, it is difficult and costly to quickly collect adequate labeled data. On the contrary, unsupervised methods do not require labeled data, but obtain a relatively lower accuracy. To balance the issues between accuracy and efficiency, we adopt a two-step identification method, including semi-automatic mode and fully automatic mode.

Semi-automatic mode: In the initial stage, Alert2vec employs a semi-automatic mode to address the lack of sufficient labeled data. This approach leverages DBSCAN (Density-Based Spatial Clustering of Applications with Noise) [11], a classic density-based clustering algorithm, to group alert vectors with similar features in high-dimensional space. The workflow consists of three key steps: (i) Clustering: well-expressed alert vectors are fed into the DBSCAN module, which identifies arbitrarily shaped clusters while filtering out noise. (ii) Manual Labeling: Security analysts perform a lightweight review, labeling only 1–2 representative alerts per cluster (e.g., high/medium/low). (iii) Label Propagation: The system generalizes these labels to all alerts within each cluster.

The semi-automatic mode can help Alert2vec reduce manual labeling effort through clustering-based sampling. It enables rapid construction of a preliminary labeled dataset, facilitating subsequent supervised learning. Besides, for those organizations with enough labeled data, Alert2vec can bypass the semi-automatic phase and proceed directly to fully supervised learning. This adaptability makes the system suitable for organizations at varying maturity levels.

Fully automatic mode: Once the data collection is finished, Alert2vec can be run entirely automatically. Considering the instability and high computational cost of deep learning methods, an acclaimed machine learning method, Random Forest, is employed to perform accurate threat level classification without any labor.

Random forest significantly enhances the generalization capability and stability of classification models by aggregating predictions from multiple decision trees. It is robust against overfitting, tolerant to noise, and capable of parallel training without requiring extensive parameter tuning. Due to these advantages, its overall performance surpasses that of most single-classifier approaches. In the field of network security, random forest has been successfully applied to various classification tasks, such as traffic analysis [12], malicious behavior analysis [13], and phishing website identification [14]. Given its proven effectiveness, Alert2vec adopts random forest as the classifier model in the fully automatic mode. With the aid of fully automatic Alert2vec, the security team can significantly reduce the workload of routine alert analysis.

IV. Subgraph Learning

In this section, we aim to construct the function H that transfers the alert subgraph A_i into a numerical vector V_{A_i}, as referred to in Definition 3. Since it is a core issue of this work, we will first introduce design intuitions and then elaborate on the specific approach in the following parts.

A. Design Intuitions

The function H needs to map alert subgraphs into distinguished vectors for identifying their threat levels, so it is obligated to discern features that declare the severity of alerts. To this end, we conduct a thorough observation on alert subgraphs from local and global perspectives.

1) Local Perspective: First, we focus on the features of the alert subgraph in a local perspective. In Fig. 6, there are four examples of the host-level and network-level alert subgraphs.

Intuition 1 (Composition): Alert subgraph's inner node and edge information may indicate the threat level. For instance, alert (a) is labeled as "Critical" because it exploits two *cmd* commands successively and causes a *blue screen of death*. Meanwhile, though generated due to triggering the static match rule to keyword "wscript.exe", alert (b) only contains an administrator subject with a startup action, which means the normal operation is falsely alerted.

Inspired by these two observations, the composition features within the alert need to be taken into account. To catch the node information, we introduce a property ***Local Neighborhood*** to represent all nodes of alert subgraphs. Likewise, the property ***Local Structure*** is proposed to catch the edge information, i.e., the correlation among alert inner elements.

Intuition 2 (Heterogeneity): The combination of heterogeneous nodes can also describe partial information of threat level. For example, in Fig. 6, both alert (c) and alert (d) have five nodes. In alert (c), a subject connects to multiple objects, which is a potential botnet marked "medium". But in alert (d), there are just one object and multiple subjects, which is a typical DDoS attack with a high threat level. Hence, we add a property ***Heterogeneity*** to express the proportions of three kinds of elements inside the alert subgraph and how they are connected.

2) Global Perspective: In addition to the internal features of alert subgraphs, the correlation between alerts also reveals their threat levels. Fig. 7 depicts a host-level AIG from the host-level dataset presented in Section V-A, which is produced by an international financial institution. Alerts of different threat levels are marked with distinct colors. The following intuitions can be derived from this figure:

Intuition 3 (Aggregation): Alert subgraphs with the same threat level are prone to cluster. In Fig. 7, there are five distinct clusters with various threat levels, i.e., areas C_1, C_2, C_3, C_4, and

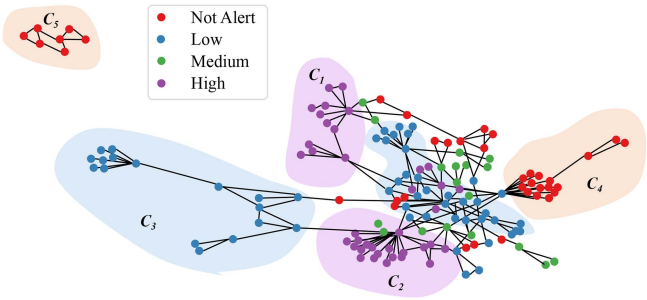


Fig. 7. An example of host-level AIG from the host-level dataset produced by an international financial institution. Alerts are categorized by threat level using a chromatic scheme: false positives (red), low-threat (blue), medium-threat (green), and high-threat (purple). The figure demonstrates clustering patterns, with alerts of similar threat levels aggregating into five distinct clusters (C1-C5).

C_5. For the adversaries, they tend to attack the same significant target (e.g., *passwd* file) and even conduct similar actions to achieve it, since the exploit of the common vulnerability is widely circulated. Hence, it results in that alert subgraphs with the same threat level are connected by a shared object or action node, like areas C_1, C_2, C_3. Besides, false alerts also tend to cluster. False alerts are mainly caused by sensitive actions from a privileged user, usually the administrator. Moreover, these sensitive actions occur successively when the administrator performs the regular inspection. Therefore, it contributes to the connection among partial false alerts through a shared subject node, like areas C_4 and C_5.

In order to determine whether an alert subgraph belongs to a certain cluster and the threat level of the cluster, it is essential to obtain information about its surrounding alert subgraphs in the AIG. Consequently, we leverage the property ***Global Neighborhood*** to express the surrounding node information and the property ***Global Structure*** to indicate the surrounding edge information.

Intuition 4 (Position): The lower the threat level of an alert, the farther it is away from the malicious alert cluster. For example, in Fig. 7, the medium-level threat cluster C_3 directly connects to the high-level cluster C_2, but the false alert cluster C_4 (with the mark of "Not alert") does not, and C_5 even becomes an island. Thus, we need a property ***Position*** to measure the distance between a subgraph and other clusters in the AIG.

To sum up, by inspecting the composition of alert subgraphs and the correlation between various alert subgraphs, we found six properties that are significantly related to alert threat levels, including local neighborhood, local structure, heterogeneity, global neighborhood, global structure, and position.

B. Local Properties

Local Neighborhood v_{Ln} is leveraged to represent an alert subgraph A_i's inner node information. To capture the node features inside the alert subgraph, we adopt an acclaimed graph learning method, node2vec [15], to generate each node's embeddings. Node2vec can learn low-dimensional representations for nodes in a homogeneous graph by optimizing a neighborhood-preserving objective. Due to the superiority of node2vec, it is also utilized when the pre-trained node embedding is demanded in the following properties.

Thus, we first compute a pre-trained node embedding set through node2vec. Since the number of alert subgraphs' nodes is not fixed, it will lead to unsolvable variable-length vector calculations if the local neighborhood v_{Ln} directly concatenates all node embeddings inside alert subgraph A_i. Hence, v_{Ln} is described as the average embedding of all nodes' embeddings in the subgraph, as the following equation exhibits.

$$v_{Ln} = \frac{1}{n}\sum_{k=1}^{n} z_k, z_k \in S_A^{(i)} \tag{2}$$

where n is the size of A_i's node set $S_A^{(i)}$, and z means the pre-trained node embedding through node2vec.

Local Structure v_{Ls} is applied to indicate the inner edge information of an alert subgraph A_i. To catch the connection features among the alert subgraph's node, we adopt a graph learning method named Anonymous Walking Embedding (AWE) [16] to derive the subgraph's local structure property. AWE can assign a low-dimensional vector representation to each graph through anonymous random walking in the graph. With the aid of AWE, we can obtain a graph embedding that represents internal path information without the influence of node type. Thus, v_{Ls} is the output of the AWE method, as the following equation states.

$$v_{Ls} = AWE(A_i) \tag{3}$$

Heterogeneity v_{Lh} is designed for expressing the node type information in A_i, since the number of heterogeneous nodes can partially indicate the threat level, as detailed in Intuition 2. To pursue v_{Lh}, first, it is essential to strive for the node embedding that can represent its element type. Therefore, we perform a supervised node classification on AIG, where labels are the element type, i.e., subject, action, or object. With the combination of node2vec and random forest, we get the node embeddings that can sufficiently express their element type. Then, considering all the three element types may potentially contribute to the threat level, v_{Lh} is organized by the concatenation of each element type's cumulative embeddings, as shown below.

$$v_{Lh} = \left[\sum_{i}^{s_n} v_s^{(i)}, \sum_{j}^{a_n} v_a^{(j)}, \sum_{k}^{o_n} v_o^{(k)}\right] \tag{4}$$

where s_n, a_n, o_n are the number of subjects, actions, and objects, respectively. v denotes the node embedding with element label.

C. Global Properties

Global Neighborhood v_{Gn} is utilized to express the surrounding node information of a given AS A_i on AIG. It can be decomposed into two steps, finding the surrounding nodes of A_i and computing v_{Gn} through these nodes.

First, a neighbor node set $S_G^{(i)}$ is defined as the union of n-hop neighbors of each border node. The border node is in A_i but connected to the area outside A_i. Generally, n less than three is adequate. Fig. 8 depicts a 2-hop neighbor node set of a given A_i, where the subject node u_1 and the action node u_2 are border nodes. Next, considering that v_{Gn} stands for the node

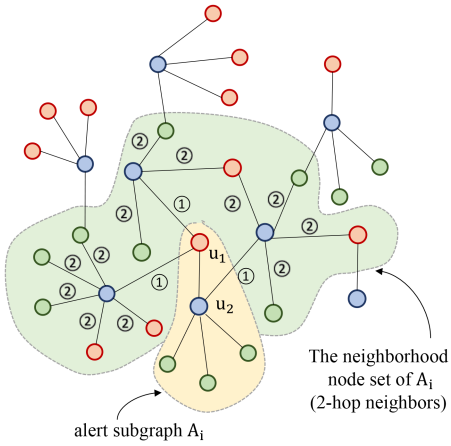


Fig. 8. An example of a 2-hop neighbor node set for a given alert subgraph A_i in AIG. The yellow area represents alert subgraph A_i and the green area represents its 2-hop partial complement graph $\bar{A}_i$. Nodes colored red are the subject elements, blue ones map to the action elements, and green ones stand for the object elements.

information, like local neighborhood v_{Ln}, v_{Gn} can be derived in a way similar to v_{Ln}, namely the average embedding of all neighbor nodes' embeddings as (5) presents.

$$v_{Gn} = \frac{1}{m}\sum_{k=1}^{m} z_k, z_k \in S_G^{(i)}, \tag{5}$$

where m is the size of neighbor node set $S_G^{(i)}$. The embedding of neighbor node z is also computed by pre-training AIG through node2vec.

Global Structure v_{Gs} is devised for storing the surrounding connection information of A_i. First, we build the partial complement graph $\bar{A}_i$, i.e., a subgraph composed of all nodes in the neighbor set $S_G^{(i)}$ and their connections on AIG. In Fig. 8, the graph delineated by the green background is an example partial complement graph. Then, v_{Gs} of A_i is also computed by AWE on $\bar{A}_i$, since AWE can provide an embedding that purely expresses the graph's connection information irrelevant to the node information. Thus, v_{Gs} is computed as follows.

$$v_{Gs} = AWE\left(\bar{A}_i\right) \tag{6}$$

where $\bar{A}_i$ is the the partial complement graph of A_i.

Position v_{Gp} is adopted to measure the distance between alert subgraph A_i and other parts on AIG. We introduce the *anchor subgraph* as the global frame of reference, which is randomly generated subgraphs from a AIG. Thus, this question can be solved by two processes, namely, randomly generating several anchor subgraphs and computing the distance between the alert subgraph and these anchor subgraphs.

The anchor subgraphs should be widely distributed on AIG and cannot overlap, which is generated randomly based on Algorithm 1. Line 6-10 selects a random seed node v, then a new anchor subgraph composed of v and its neighbors is produced by line 11-16. The anchor rate $p \in (0, 0.1]$ in line 4 controls the number of anchor subgraphs and is generally set to 0.03.

Algorithm 1: Generate Anchor Subgraphs.

Input: Alert Intelligence Graph G, anchor rate p
Output: Set A_S composed of anchor subgraphs
1: Initialize set V to the node set of graph G.
2: Initialize set A_S to an empty set.
3: Let n as the number of nodes in G.
4: $a_n = n * p$.
5: **while** $(V \neq \phi)$ and $(len(AS) \leq a_n)$ **do**
6: Select a node v in V randomly.
7: **while** $(V \neq \phi)$ and (v's neighbor has been selected) **do**
8: $V = V - v$.
9: Reselect a node v in V randomly.
10: **end while**
11: Let V_a as the set of all neighbor nodes of v.
12: $V_a = V_a + v$
13: Let E_a as edges between nodes from V_a in G.
14: Let G_a as the graph composed of V_a and E_a.
15: $A_S = A_S + G_a$.
16: $V = V - V_a$.
17: **end while**
18: **return** A_S

Subsequently, the embedding of each anchor subgraph Z_a can be acquired by aggregating its nodes, such as the average of all node embeddings pre-trained by node2vec. Finally, the position property is formalized as the following equation:

$$v_{Gp} = \frac{1}{n}\sum_{i}^{n} \frac{1}{d_i} \cdot Z_a^{(i)} \tag{7}$$

where n is the number of anchor subgraphs, d_i means the average of the shortest path's length from each node in A_i to the anchor subgraph.

D. Subgraph Representation

Finally, the above six properties are concatenated to build the representation of alert subgraph v_{A_i}, as formulated in (8).

$$v_{A_i} = [v_{Gn}, v_{Gs}, v_{Gp}, v_{Ln}, v_{Ls}, v_{Lh}] \tag{8}$$

It is noted that the dimensions of all properties are not equal exactly since they make various contribution to identifying the threat level. The best proportion of properties will be discussed in Section VI. Furthermore, the subgraph learning approach is flexible. Namely, both node2vec for generating node embedding and AWE for capturing structural information can be replaced by other similar methods according to the requirements when extended to different working scenarios.

V. Experimental Settings

A. Datasets

To evaluate the practical efficacy of Alert2vec, we performed experiments on three real-world datasets: two private datasets—the **Host-Level Dataset (HD)** and the **Network-Level Dataset (ND)**—and one publicly available dataset, **NIDS-Alert-Data**

TABLE I
THE STATISTICAL INFORMATION OF DATASETS

	Network-level (ND)	Host-level (HD)	Nids-Alert-Data (NAD)
Alerts	17306	230	1395325
Subject types	5593	46	45339
Action types	39	42	587
Object types	4130	78	4402
Threat levels	3	4	2
False alerts	/	27.8%	98.5%
Low alerts	46.6%	53.9%	/
Medium alerts	50.5%	3.9%	/
Hgih alerts	2.9%	14.3%	1.5%

(NAD) [17]. Their statistical information is exhibited in Table I. HD collects all alerts of security devices deployed on hosts from the Asian Development Bank (ADB) for two months. Owing to the careful configuration, there are 230 alerts in total. In contrast, ND contains alerts from security devices deployed in the Tsinghua University's network during a week. Since the strict detection policy and the large scale of this network, there are over 10 thousand alerts. For both datasets, the threat levels are labeled by detection systems and security analysts. NAD contains over 1.3 million labeled network intrusion detection system (NIDS) alerts, grouped by external IP and signature, collected over 60 days from a large institution's perimeter. The threat levels of alerts in NAD are manually assessed and classified by security analysts into two categories: important and irrelevant.

B. Baselines

To prove the superiority of Alert2vec in the task of alert threat-level identification for various security devices in a large organization network, we compare Alert2vec with the following baselines.

- *Rule-based:* It leverages several manually preset filtering rules for classification, which is used to simulate the usual processing of security workers in the real world.
- *AlertRank [2]:* it is a scene-specific method designed for identifying severe alerts for online service systems through XGboost and LSTM.
- *Context2vector [5]:* This method constructs various alerts into a security event sequence and computes an anomalous score for each alert based on the contextual bias to find critical alerts.
- *DeepCase [4]:* Likewise, it is also an event-sequence-based alert analysis method but stresses aggregating similar alerts.
- *StreamCluster [18]:* It proposes a hybrid approach combining stream clustering and supervised learning to effectively classify alerts generated by network intrusion detection systems.
- *ActiveLearning [19]:* It explores the use of active learning techniques to enhance the accuracy and efficiency of network intrusion detection system alert classification.

Besides, to valid the effectiveness of our subgraph learning algorithm for threat-level identification, we compare Alert2vec with the following subgraph representation methods.

- *Structure2vec [6]:* Since Structure2vec is designed to generate node embeddings, we aggregate the subgraph's node embeddings as the representation.
- *Metapath2vec [20]:* Likewise, Metapath2vec is used for producing the node embedding of heterogeneous graph. The subgraph vector is composed of the average embedding of its nodes.
- *Sub2vec [7]:* Sub2vec extends Paragraph2vec [21] to learn subgraph embeddings and proposes two branches distinguished by the properties used, **Sub2vec-N**(using neighborhood property) and **Sub2vec-S**(using structural property). Besides, we merge embeddings from Sub2vec-N and Sub2vec-S as a new baseline named **Sub2vec-NS**.
- *SubGNN [8]:* It is the state-of-the-art subgraph learning method that builds a subgraph neural network based on three kinds of channels.

To measure the effectiveness of alert embeddings yielded from the above methods and Alert2vec, we leverage a supervised classifiers, RF, and a clustering method, DBSCAN, to conduct vector classification. Besides, accuracy, recall and f1-score are applied as metrics to quantify performance. In order to ensure the reliability, each result is the average value after three repetitions, and its standard deviation is indicated after the sign "±".

C. Implementation Details

Alert2vec is implemented on python 3.6 with packets of *Sklearn* and *Networkx*. When adopting supervised methods to distinguish various alert embeddings, 70% of the data is utilized for training the classifier, and the left data is used for testing. In the fully automatic mode, the random forest classifier adopts the following configuration: n_estimators=1000 (to ensure robust ensemble learning), criterion='gini' (for impurity-based splitting), max_depth=None (allowing unconstrained tree growth), min_samples_split=2, min_samples_leaf=1 (minimal splits for fine-grained partitioning), and max_features='sqrt' (square-root-sampled features per split). For the semi-automatic mode, DBSCAN is parameterized with eps=0.30 (neighborhood radius), min_samples=9 (core-point threshold), metric='euclidean' (distance measure), and leaf_size=30 (optimized nearest-neighbor search efficiency).

In addition, AlertRank cannot be fully reproduced in implementation. The full version of AlertRank includes 12 kinds of features with a total of 40 dimensions. Due to its scene-specific feature engineering, KPI anomaly features (containing 13-dimensional features) are missing in all datasets. Therefore, only 27-dimensional features can be leveraged, compared to the full version.

Furthermore, an additional step is added to make DeepCase perform threat-level identification since DeepCase is merely designed for aggregating similar alerts initially. Namely, it is necessary to manually determine the threat level of the resulting similar alert clusters, just like the process of the semi-automated version of Alert2vec.

VI. EXPERIMENT RESULTS

In this section, we aim to answer the following questions.

Q1 Does Alert2vec have better performance in identifying threat levels of alerts?

Q2 Does Alert2vec propose a better subgraph learning method for generating alert embedding?

TABLE II
ACCURACY COMPARISON BETWEEN ALERT2VEC AND OTHER BASELINE METHODS

	ND	HD	NAD
Rule-based	0.476±0.033	0.481±0.029	0.748±0.011
AlertRank	0.632±0.041	0.671±0.021	0.825±0.047
Context2vector	0.753±0.019	0.819±0.027	0.708±0.042
DeepCase	0.568±0.021	0.594±0.023	0.621±0.020
StreamCluster	0.473±0.029	0.809±0.018	0.980±0.028
ActiveLearning	0.480±0.019	0.347±0.003	0.932±0.023
Alert2vec	**0.947±0.014**	**0.956±0.016**	**0.991±0.007**

TABLE III
THE TESTING ACCURACY FOR DIFFERENT PROPERTIES

	ND	HD
local neighborhood	0.910±0.029	0.927±0.051
global neighborhood	0.914±0.024	0.931±0.032
local structure	0.502±0.014	0.670±0.055
global structure	0.506±0.023	0.702±0.063
position	0.781±0.081	0.786±0.077
heterogeneity	0.775±0.024	0.916±0.047
all properties	**0.947±0.014**	**0.956±0.016**

Q3 How much can each property contribute to the result?
Q4 How to allocate proportions of properties in v_{A_i}?

Q1: Does Alert2vec have better performance in identifying threat levels of alerts?

To evaluate the effectiveness of Alert2vec in assessing security alert threat levels, we compared its classification accuracy with baseline methods (Table II). Alert2vec achieved superior performance across two private datasets (ND: 94.7%; HD: 95.6%) and one public dataset (NAD: 99.1%), demonstrating superior applicability. The following section analyzes these results in comparison with baseline models.

First, it reveals that the rule-based method exhibit limited effectiveness, as their static rules are only applicable to a subset of alerts. Furthermore, when comparing AlertRank with Alert2vec, it can be derived that single-scenario methods underperform in cross-scenario settings due to insufficient scenario-specific feature representation and excessive model specialization. Then, to further demonstrate Alert2vec's superiority in graphically representing alert correlations, we compared it with Context2vec and DeepCase. As event sequence-based methods, both DeepCase and Context2vec can only capture limited linear correlations from chronologically ordered alerts. Moreover, DeepCase underperforms Context2vec due to its focus on aggregating similar security events rather than classifying alert threat levels. Finally, we evaluate Alert2vec's versatility through comparison with two state-of-the-art alert classification methods: StreamCluster and ActiveLearning. While these methods employ clustering and active learning techniques respectively to optimize random forest classifiers using small-scale training datasets (thereby reducing dependency on labeled samples), both remain heavily reliant on manual feature engineering. Consequently, although they achieve strong performance on the NAD dataset, their effectiveness significantly degrades when applied to other scenarios (ND and HD).

Unlike baseline methods, Alert2vec's graph-based framework captures both alert-level and element-level correlations, achieving superior effectiveness and versatility.

Q2: Does Alert2vec propose a better subgraph learning method for generating alert embedding?

To measure the effectiveness of our customized subgraph representation method, we replace it with other subgraph learning baselines mentioned in Section V-B, and compare the final performance. As Section III-E stated, we first run the model in a semi-automatic mode to simulate the accuracy during the start-up process of collecting labeled data. Then run the model in a fully automatic mode to pursue final accuracy.

The overall results of all baselines and Alert2vec are shown in Fig. 9. Alert2vec outperforms other baselines on both datasets, with the best accuracy of 94.7% on ND and 95.6% on HD. Fully automatic Alert2vec gains the best accuracy since it commendably leverages the features of all properties. Semi-automatic Alert2vec performs inevitably worst as an unsupervised method, but it still plays a vital role in quickly collecting labelled data when adopting in the real world.

Compared to the first two methods, Structure2vec and Metapath2vec, it indicates that simply aggregating node embeddings as the subgraph vector is not available, regardless of the complexity of the node embedding. Though, in addition to neighborhood information, Structure2vec and Metapath2vec consider nodes' surrounding structure and heterogeneous information respectively, they still lose abundant local and global features of subgraphs.

Results on three kinds of Sub2vec models reveal that partial information of subgraphs is not adequate for classification. Sub2vec-N preserves the local neighborhood property for subgraph, while Sub2vec-S extracts the local structure property. Then, Sub2vec-NS concatenates embeddings from these two models. Due to the lack of global and heterogeneous information, these methods cannot competently determine the threat level of alert subgraphs.

In comparison with the best model, SubGNN, Alert2vec has an accuracy improvement of 4% on HD and 3% on ND. Although SubGNN considers structure, neighbor, and position properties for subgraph representations, the computation way of these properties is not proper for alert subgraphs. On the one hand, SubGNN cannot incorporate the alert subgraph's heterogeneous information into the representation. On the other hand, it is mainly devised for non-trivial subgraphs, focusing on the remote information when computing global neighborhood and structure properties. However, the surrounding information is more vital for alert subgraphs as introduced in Section IV.

Q3: How much can each property contribute to results?

To investigate the influence of each property when generating the alert representation, we conduct several experiments, as Table III exhibits. Each time only a vector generated by one property is used to classify the threat level through random forest.

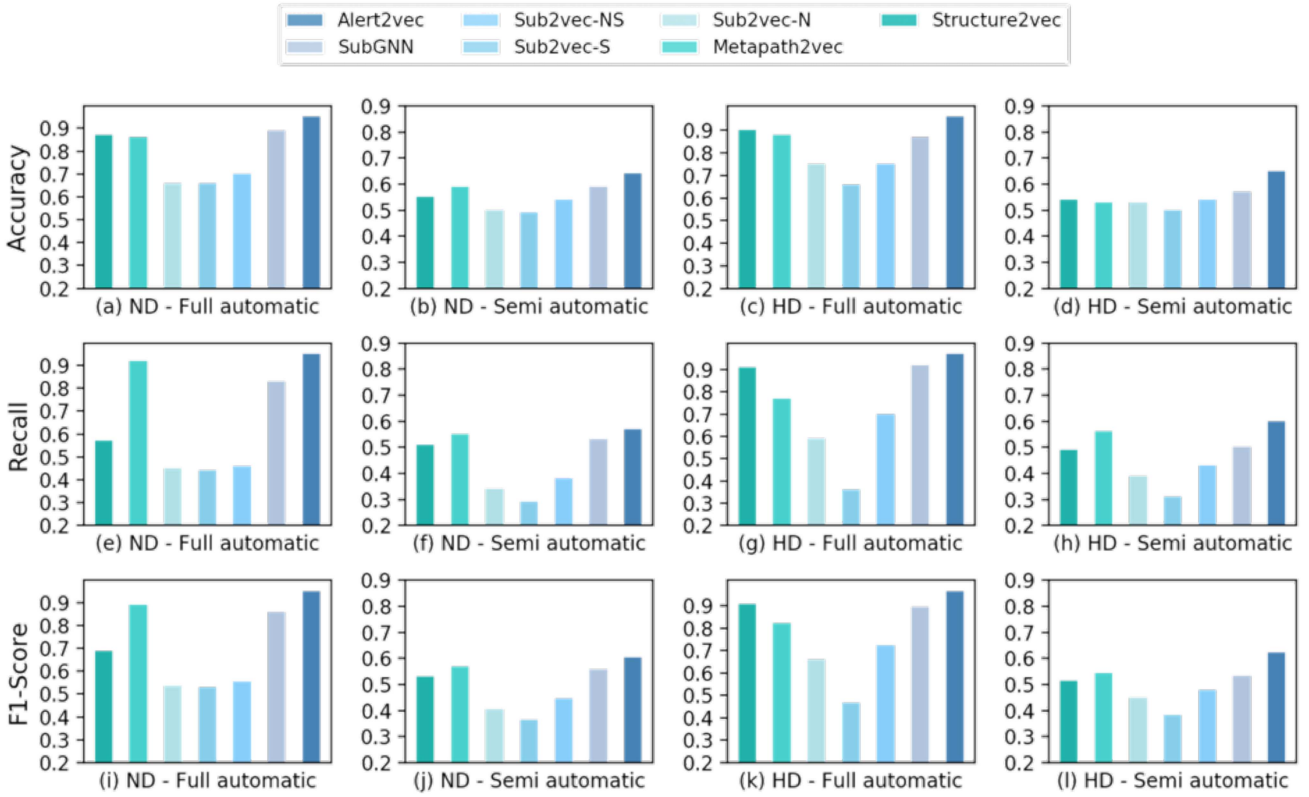


Fig. 9. Performance comparison between Alert2vec and other subgraph learning methods, in terms of accuracy, recall, and f1-score.

In the first two experiments, the global and local neighborhoods achieve relatively higher accuracy than other properties, about 91% on the ND and 92.7% on HD. It declares the significance of surrounding and inner nodes' information to identify threat levels for an alert subgraph, since alerts with the same level are prone to cluster and even partially overlap, as stated in Section IV. In contrast, the global and local structures provide the worst performance due to the limited capability for graph structure to express distinguished features. However, results on HD are better than those on ND. It is attributed to the higher complexity of host-level alert subgraphs. Alerts in the host scenario can be triggered by multiple behaviors, causing more complicated subgraph structures, which tends to be a high-level threat.

Moreover, although results of position and heterogeneity are not the best, it still demonstrates their effectiveness in identifying threat levels. The position property can measure the distance between the alert subgraph and other potential malicious clusters on AIG, which is a vital feature mentioned in Section IV. As for heterogeneity, it can reveal the differences in the composition of heterogeneous elements within the subgraph. Especially on HD, it shows the superiority for identifying threat levels of complex alert subgraphs.

Finally, the best accuracy on all properties, namely, the complete Alert2vec, demonstrates the essentiality of combining the above six properties to perform alert classification. What's more, the complete Alert2vec can provide more stable performance with the slightest standard deviation.

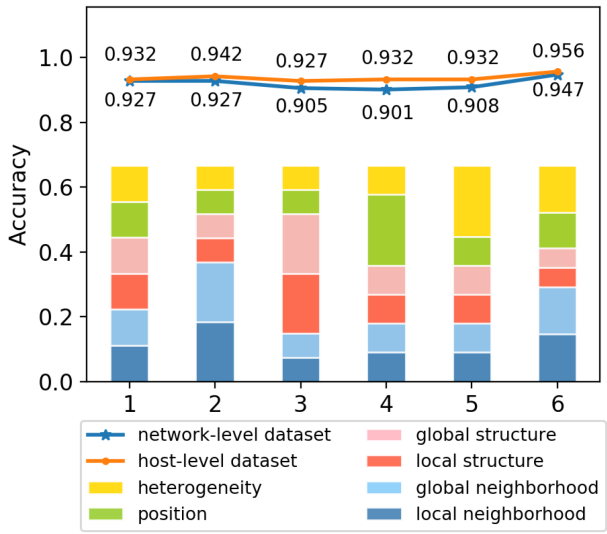


Fig. 10. The accuracy results of various dimension settings. The area of each colored bar represents the proportion of the corresponding property's dimension in the alert vector.

Q4: How to allocate proportions of properties in v_{A_i}?

Since each property may contribute to the result to a varying extent, vector dimensions of all properties should be adjusted according to their weights. To pursue a proper setting, we conduct six experiments as Fig. 10 depicts. The proportion of each property is measured as the colored bar's area.

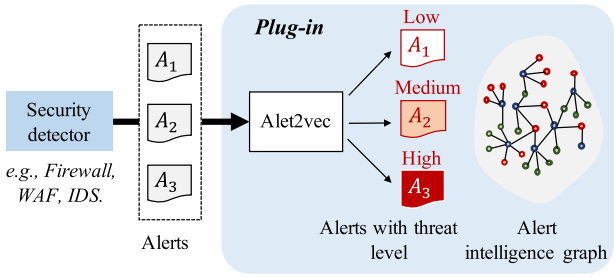


Fig. 11. Alert2vec operates as a plug-and-play module behind security devices, it can processe security alerts as subgraphs and analyzes threat levels via alert intelligence graphs.

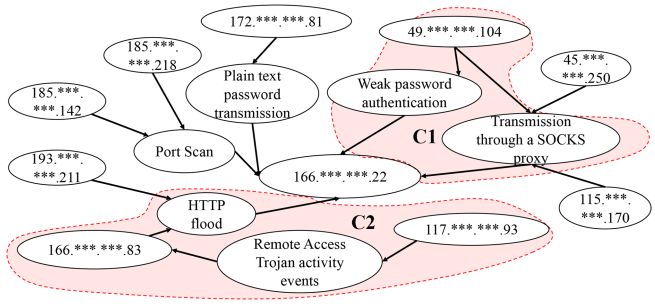


Fig. 12. A practical example of an AIG derived from the ND dataset. The AIG aggregates alerts centered on the victim IP *166.***.***.22*. Alerts in the red-labeled regions **C1** and **C2** are classified as *high-risk*, while others are labeled *medium-risk* or *low-risk*.

The first experiment is a benchmark, where the proportion of all properties is equal. The performance on HD increases slightly when the proportion of global and local neighborhood properties is increased to 56% totally. While the accuracy decreases when raising the weight of global and local structure properties, the position property, the heterogeneity property in experiment 3, 4, 5, respectively. It indicates that the neighborhood information can provide more valid features for classification.

The best performance is presented in the final experiment, where dimensions of v_{Gn}, v_{Ln}, v_{Gs}, v_{Ls}, v_{Gp}, v_{Lh} are 128, 128, 52, 52, 96, 128, respectively. The comparison of experiment 6 and experiment 2 demonstrates that too large proportion for neighborhood information may skew the classifier, leading to misjudgment in the condition that other information is essential to determine the threat level. In contrast, the dimensions setting in experiment 6 can sufficiently integrate the valid information of each property.

VII. Discussion

A. Extension of Alert2vec

Alert2vec can be used as a plug-in attached to different security devices. Once an alert is produced by a security device, it can be formulated as an alert subgraph by extracting the subject, object, and action nodes, as Section III-C stated. Then, as Fig. 11 shows, alert2vec can provide the threat level of this alert and the alert intelligence graph. Thus, Alert2vec can be directly concatenated behind security devices as a plug-in. Furthermore, Alert2vec has a flexible framework. There are two fundamental steps when obtaining six properties in subgraph learning, namely, the computation of node and structural embeddings. Targeted at alert classification, our implementation of Alert2vec applies node2vec and AWE for generating the pre-trained node embedding and structural embedding, respectively. They can be replaced by other similar methods according to requirements when transferring Alert2vec to other problem scenarios.

B. Limitation

Although Alert2vec demonstrates superior performance in alert classification across multiple security devices, it has two key limitations:

i) Data Quality: The model's performance heavily relies on the quality and completeness of historical alert data, as it constructs a large-scale alert intelligence graph. Short-term or fragmented datasets may introduce sampling bias, degrading performance. For optimal results, at least one month of continuous alert data is recommended to capture recurring attack patterns (e.g., periodic scanning, weekend anomalies).

ii) Time-Series Alert Filtering Deficiency: While Alert2vec effectively correlates discrete alerts across security devices (e.g., IDS, firewalls), it struggles to suppress redundant or false alerts in time-series anomaly detection. This limitation arises because the input representation of time-series includes only timestamps t_i and indicator deviations x_i, omitting critical contextual elements (e.g., *subject*, *object*, *action*)—essential for constructing alert subgraphs and alert intelligence graphs. Consequently, Alert2vec cannot classify time-series alerts effectively. Thus, filtering alerts in time-series-based alerts (e.g., CPU usage, memory consumption) remains an open challenge.

C. Case Study

Fig. 12 presents a practical example of an alert intelligence graph derived from the ND dataset, where IP addresses are partially anonymized (e.g., *166.***.***.22*) to protect user privacy. The alert intelligence graph aggregates alerts centered on the victim IP *166.***.***.22*. Alerts in the red-labeled regions **C1** and **C2** are classified as *high-risk*, while others are labeled *medium-risk* or *low-risk*. In case **C1**, the same SOCKS proxy transmission behavior triggers a low-threat alert for IP *45.***.***.250* but a high-risk alert for IP *49.***.***.104*. The latter is attributed to its concurrent high-threat *weak password authentication* attack against the victim. Alert2vec can evaluate contextual threat relationships through correlation of attacker-associated nodes, identifying high-risk behaviors and empowering security teams to efficiently mitigate malicious IP operations. In case **C2**, an HTTP flood from IP *193.***.***.211* is labeled medium-risk, whereas the same behavior from IP *166.***.***.83* is flagged high-risk. The alert intelligence graph reveals that *166.***.***.83* was previously targeted by a high-risk *Remote Access Trojan* from IP *117.***.***.93*. By tracing attacker-node associations, Alert2vec escalates the threat assessment, reminding administrators that the attacking IP *166.***.***.83* may be a compromised

internal node needing intervention. These examples demonstrate Alert2vec's capability to correlate alert components, prioritize critical alerts, and mitigate alert fatigue for network operators.

VIII. Related Work

In this section, we described the limitations of previous works and complemented the discussion on related work here in terms of alert fatigue elimination, alert correlation, and subgraph representation.

A. Eliminating Alert Fatigue

In order to combat alert fatigue, many efforts have been made in various scenarios. Recently, Zhong et al. [22] mined previous' operation traces to construct a state machine for alert classification on IDS and firewall logs. Laszka et al. [23] extended strategic threshold-selection work to mitigate the threat posed by spear-phishing attacks. Hassan et al. [1] leveraged system provenance graph to conduct single-host level alert triage process. Zhao et al. [2] adopted multiple scene-specific features to identify severe alerts for online service systems, such as server KPI. Hassan et al. [3] introduced tactical provenance graphs to recognize the advanced persistent related threats produced by EDR. Aminanto et al. [24] developed an alert classification method using the feature engineering approach based on the Isolation Forest, leveraging over 20 field features. Similarly, Ban et al. [25] also constructed the classification method in a feature engineering way, however, through six supervised machine learning methods, such as support vector machines. To sum up, these works used scene-specific features or methods on a given scenario, which were not sufficient to perform alert classification for heterogeneous alerts generated by various security devices in a large organizational network. To reduce the burden for the security team to a maximum extent, we propose Alert2vec, a general alert classification framework that can be applied to most of distinct security devices with heterogeneous alert reports.

B. Alert Correlation

As mentioned above, the similarity and context among alerts can help assess an alert's threat level. To capture this information, a way to correlate multiple alerts is required. There are two typical approaches, sequence-based [4], [5] [26], [27] and graph-based [1], [28], [29], [30]. The sequence-based method arranges the alerts in chronological order, and utilizes deep learning methods to capture the contextual information between alerts. For example, DeepCase [4] converted a bunch of alerts into a security event sequence and aggregated similar security events through GRU [31]. However, such methods can only present alerts in a simple linear way, losing the information of the complex interaction among elements of alerts. In contrast, the graph-based method is more appropriate for displaying nonlinear correlations among alerts since it can represent the connection between any alert's elements through edges. Nevertheless, most current works employ the provenance graph for analysis, which is only applicable to alerts generated in the host audit log and has a vast scale. To make it suitable for both host-level and network-level security devices, Alert2vec proposes a kind of alert intelligence graph that can represent the correlation of heterogeneous alerts from a fine-grained perspective and has an acceptable scale. The alert intelligence graph allows correlations among alerts to be fully exhibited.

C. Subgraph Representation

To perform the threat-level identification automatically, there need a graph learning method to characterize heterogeneous alert subgraphs. Previous works of graph learning mainly focused on node embedding [6], [32], [33] or network embedding [34], [35], which is unavailable for generating subgraph embedding. Recently, Yanardag et al. [36] and Narayanan et al. [37] conducted subgraph representation through Word2Vec by treating sub-structures in graphs as words simply, losing a lot of structure and type information of heterogeneous subgraphs. Adhikari et al. [7] built subgraph embedding based on local neighborhood and structure properties. However, they were unable to capture the global information and heterogeneous property of alert subgraphs, which cannot generate well-expressed embeddings. Meng et al. [9] performed subgraph evolution prediction on fixing-size subgraphs, which is invalid for variable-length alert subgraphs. Alsentzer et al. [8] introduced a channel-based subgraph neural network for subgraphs with non-trivial internal topology, but it cannot extract the heterogeneous node information in alert subgraphs. In short, existing subgraph learning methods are devised for data mining without specific considerations for our scenarios, making them incompetent at extracting all features of alert subgraphs. Consequently, Alert2vec proposes a customized subgraph representation approach, which can generate a well-expressed alert embedding since it catches all six properties of the alert subgraph, namely, local neighborhood, global neighborhood, local structure, global structure, position, and heterogeneity.

D. Applied Graph Learning for Cybersecurity

Graph learning encompasses machine learning models and methods specifically tailored for graph-structured data. These techniques perform prediction and classification tasks by learning the topological properties, node features, and edge relationships within graphs. Given that graph representations can effectively model the contextual relationships inherent in attack scenarios, graph learning has been increasingly adopted in cybersecurity research in recent years. For instance, in host-based intrusion detection systems, numerous studies have applied graph learning to perform deep analysis on provenance graphs constructed from system audit logs [29], [30], [38], [39], [40], [41]. Similarly, this approach has been leveraged across diverse cybersecurity domains, such as network intrusion detection [42] and attack prediction [43], demonstrating consistent improvements in task performance.

IX. Conclusion

In this paper, we present a general alert classification framework, Alert2vec, which can eliminate alert fatigue in both

network-level and host-level scenarios. Alert2vec proposed a three-element-based extraction method to formalize heterogeneous alert records produced by various security devices as alert subgraphs. Then all alert subgraphs are incorporated into an alert intelligence graph to capture their correlation. Additionally, a novel six-property-based subgraph representation method is devised to express prominent features of alert subgraphs. Moreover, Alert2vec provides a flexible two-mode classification method, supporting semi-automatic and fully automatic threat level identification. Finally, we conduct several experiments on both network-level and host-level datasets, validating the effectiveness of Alert2vec.

References

[1] W. U. Hassan et al., "NoDoze: Combatting threat alert fatigue with automated provenance triage," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, Feb. 24–27, 2019, doi: 10.14722/ndss.2019.23349.

[2] N. Zhao et al., "Automatically and adaptively identifying severe alerts for online service systems," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 2420–2429.

[3] W. U. Hassan, A. Bates, and D. Marino, "Tactical provenance analysis for endpoint detection and response systems," in *Proc. 2020 IEEE Symp. Secur. Privacy*, 2020, pp. 1172–1189.

[4] T. van Ede et al., "DEEPCASE: Semi-supervised contextual analysis of security events," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 522–539.

[5] J. Liu et al., "Context2Vector: Accelerating security event triage via context representation learning," *Inf. Softw. Technol.*, vol. 146, 2022, Art. no. 106856.

[6] L. F. Ribeiro, P. H. Saverese, and D. R. Figueiredo, "Struc2Vec: Learning node representations from structural identity," in *Proc. 23 rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 385–394.

[7] B. Adhikari, Y. Zhang, N. Ramakrishnan, and B. A. Prakash, "Sub2Vec: Feature learning for subgraphs," in *Proc. Pacific-Asia Conf. Knowl. Discov. Data Mining*, 2018, pp. 170–182.

[8] E. Alsentzer, S. G. Finlayson, M. M. Li, and M. Zitnik, "Subgraph neural networks," 2020, *arXiv: 2006.10538*.

[9] C. Meng, S. C. Mouli, B. Ribeiro, and J. Neville, "Subgraph pattern neural networks for high-order graph evolution prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 3778–3787.

[10] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[11] M. Ester et al., "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. Knowl. Discov. Data Mining*, 1996, pp. 226–231.

[12] S.-J. Xu, G.-G. Geng, X.-B. Jin, D.-J. Liu, and J. Weng, "Seeing traffic paths: Encrypted traffic classification with path signature features," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 2166–2181, 2022.

[13] M. Sharif, J. Urakawa, N. Christin, A. Kubota, and A. Yamada, "Predicting impending exposure to malicious content from user behavior," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2018, pp. 1487–1501.

[14] B. Kondracki, B. A. Azad, O. Starov, and N. Nikiforakis, "Catching transparent phish: Analyzing and detecting mitm phishing toolkits," in *Proc. 2021 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 36–50.

[15] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 855–864.

[16] S. Ivanov and E. Burnaev, "Anonymous walk embeddings," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 2186–2195.

[17] S. Ristov, "Nids alert data," 2021. [Online]. Available: https://github.com/ristov/nids-alert-data

[18] R. Vaarandi and A. Guerra-Manzanares, "Stream clustering guided supervised learning for classifying NIDS alerts," *Future Gener. Comput. Syst.*, vol. 155, pp. 231–244, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X24000396

[19] R. Vaarandi and A. Guerra-Manzanares, "Network IDS alert classification with active learning techniques," *J. Inf. Secur. Appl.*, vol. 81, 2024, Art. no. 103687. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214212623002715

[20] Y. Dong, N. V. Chawla, and A. Swami, "metapath2vec: Scalable representation learning for heterogeneous networks," in *Proc. 23 rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 135–144.

[21] Q. Le and T. Mikolov, "Distributed representations of sentences and documents," in *Proc. Int. Conf. Mach. Learn.*, 2014, pp. 1188–1196.

[22] C. Zhong, J. Yen, P. Liu, and R. F. Erbacher, "Automate cybersecurity data triage by leveraging human analysts' cognitive process," in *Proc. IEEE 2nd Int. Conf. Big Data Secur. Cloud, IEEE Int. Conf. High Perform. Smart Comput.*, 2016, pp. 357–363.

[23] A. Laszka, J. Lou, and Y. Vorobeychik, "Multi-defender strategic filtering against spear-phishing attacks," in *Proc. AAAI Conf. Artif. Intell.*, 2016, pp. 537–543.

[24] M. E. Aminanto, L. Zhu, T. Ban, R. Isawa, T. Takahashi, and D. Inoue, "Combating threat-alert fatigue with online anomaly detection using isolation forest," in *Proc. Neural Inf. Process.: 26th Int. Conf.*, Sydney, NSW, Australia, 2019, pp. 756–765.

[25] T. Ban, N. Samuel, T. Takahashi, and D. Inoue, "Combat security alert fatigue with AI-assisted techniques," in *Proc. Cyber Secur. Experimentation Test Workshop*, 2021, pp. 9–16.

[26] S. Wu, B. Wang, Z. Wang, S. Fan, J. Yang, and J. Li, "Joint prediction on security event and time interval through deep learning," *Comput. Secur.*, vol. 117, 2022, Art. no. 102696.

[27] Y. Shen, E. Mariconti, P. A. Vervier, and G. Stringhini, "Tiresias: Predicting security events through deep learning," in *Proc. 2018 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 592–605.

[28] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. Venkatakrishnan, "POIROT: Aligning attack behavior with kernel audit records for cyber threat hunting," in *Proc. 2019 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1795–1812.

[29] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "UNICORN: Runtime provenance-based detector for advanced persistent threats," 2020, *arXiv: 2001.01525*.

[30] S. Wang et al., "THREATRACE: Detecting and tracing host-based threats in node level through provenance graph learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 3972–3987, 2022.

[31] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," 2014, *arXiv:1406.1078*.

[32] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 701–710.

[33] D. Wang, P. Cui, and W. Zhu, "Structural deep network embedding," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1225–1234.

[34] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "LINE: Large-scale information network embedding," in *Proc. 24th Int. Conf. World Wide Web*, 2015, pp. 1067–1077.

[35] X. Wang, P. Cui, J. Wang, J. Pei, W. Zhu, and S. Yang, "Community preserving network embedding," in *Proc. 31st AAAI Conf. Artif. Intell.*, 2017, pp. 203–209.

[36] P. Yanardag and S. Vishwanathan, "Deep graph kernels," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2015, pp. 1365–1374.

[37] A. Narayanan, M. Chandramohan, L. Chen, Y. Liu, and S. Saminathan, "Subgraph2Vec: Learning distributed representations of rooted sub-graphs from large graphs," 2016, *arXiv:1606.08928*.

[38] M. U. Rehman, H. Ahmadi, and W. U. Hassan, "Flash: A comprehensive approach to intrusion detection via provenance graph representation learning," in *Proc. IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, 2024, pp. 3552–3570, doi: 10.1109/SP54263.2024.00139.

[39] F. Yang, J. Xu, C. Xiong, Z. Li, and K. Zhang, "PROGRAPHER: An anomaly detection system based on provenance graph embedding," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 4355–4372.

[40] E. Manzoor, S. M. Milajerdi, and L. Akoglu, "Fast memory-efficient anomaly detection in streaming heterogeneous graphs," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1035–1044.

[41] J. Zengy et al., "SHADEWATCHER: Recommendation-guided cyber threat analysis using system audit records," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 489–506.

[42] X. Hu, W. Gao, G. Cheng, R. Li, Y. Zhou, and H. Wu, "Towards early and accurate network intrusion detection using graph embedding," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 5817–5831, 2023.

[43] Y. Liu and Y. Guo, "CL-AP2: A composite learning approach to attack prediction via attack portraying," *J. Netw. Comput. Appl.*, vol. 230, 2024, Art. no. 103963. [Online]. Available: https://api.semanticscholar.org/CorpusID:271036299

Songyun Wu received the BE degree in information security from Beijing Jiaotong University, Beijing, China, in 2019, and the PhD degree in cybersecurity from Tsinghua University, Beijing, China, in 2025. She is currently an Assistant Research Professor with the Zhongguancun Laboratory, Beijing. Her research interests include network security and applied machine learning.

Xiaoqing Sun received the BS degree from the Beijing University of Posts and Telecommunications, in 2016 and the PhD degree from the Department of Computer Science and Technology, Tsinghua University, in 2023. She is currently a researcher in Alibaba Cloud. Her reseaerch interests include Cloud Computing and Cybersecurity.

Enhuan Dong received the BE degree from the Harbin Institute of Technology, Harbin, China, in 2013, and the PhD degree from Tsinghua University, Beijing, China, in 2019. He was a visiting PhD student with the University of Goettingen, in 2016-2017. He is currently an assistant research professor in the Institute for Network Sciences and Cyberspace, Tsinghua University. His research interests include network security, network operations and network transport.

Zhiliang Wang received the BE, ME, and PhD degrees in computer science from Tsinghua University, China, in 2001, 2003, and 2006 respectively. Currently he is an associate professor in the Institute for Network Sciences and Cyberspace with Tsinghua University. His research interests include network formal verification and testing, network architecture and protocols, network measurement and monitoring.

Chen Zhao is the Head of Security Engineering and Operations in Asian Development Bank (ADB). Chen Zhao and his team are responsible for IT Security infrastructure engineering, monitoring and incident response activities at ADB. Prior to joining ADB, he led security operations at venture capital firm Sequoia Capital. He started his career 14 years ago as a cyber security consultant in PwC Hong Kong and US, providing security and risk services for Fortune 100 companies in several continents.

Jiahai Yang (Senior Member, IEEE) received the BSc degree in computer science from Beijing Technology and Business University, and the MSc and PhD degrees in computer science from Tsinghua University, Beijing, China. He is currently a professor with Institute for Network Sciences and Cyberspace, Tsinghua University. His research interests include network management, network measurement, network security, Internet routing, cloud computing, and Big Data applications.